Lecture plan · for the lecturer

Hashing, passwords and authentication

Cyber security · University · Lecture 3

Time
60 min

Materials
Projector or board; The lecture slides and the seminar sheet from the kit; Phones or laptops with Darsi, for the lab

Learning goals

- State hash properties.
- Store passwords properly.
- Choose a second factor.

Activity: Crack a fake table

1. Warm-up (5 min): Ask students to guess how many SHA-256 hashes one GPU computes per second, then reveal the order of magnitude.
2. Teach (10 min): Show the avalanche effect and the three resistance properties.
3. Crack (25 min): Teams run a dictionary attack on fast, salted and Argon2id tables of synthetic users.
4. Decide (15 min): Write a one-paragraph storage and MFA policy for the university.
5. Wrap-up (5 min): Students write the muddiest point of the lecture and one question; read them before next week.

Discussion

- How many accounts fell in each table?
- Why do salts matter even with slow hashes?
- Which factor would you require for staff?

Exit ticket

- Write the storage line you would use for one password.

Try it in your next lecture
Give teams a small table of fake users hashed three ways; they time a dictionary attack on each and explain the difference.

Seminar sheet · for students

Hashing, passwords and authentication

Cyber security · University · Lecture 3

Name _____ Group _____ Date _____

1 **A lecturer signed in through a fake page that passed everything she typed or approved to the real site at once. Which second factor would have stopped the attacker?**

◯ A push prompt to approve the sign-in ◯ A FIDO2 security key ◯ An authenticator-app code

2 **Estimate an attacker's work.**

A cracker tries 10^9 fast hashes per second. If each Argon2id guess costs 10^6 times more, it manages _____ guesses per second. A 10^7-word dictionary then takes _____ seconds per account.

Word bank: 1 · 1000 · 10000 · 100

3 **Which are sound password-storage practices? Choose all that apply.**

☐ SHA-256 with one salt for all users ☐ Checking new passwords against breached lists

☐ Argon2id with a unique salt per user ☐ Encrypting passwords so admins can read them

4 **A real-time phishing proxy can relay a one-time code sent by SMS before the code expires.**

◯ True ◯ False

5 **Match each attack to the defence that most directly stops it.**

Precomputed rainbow tables	• • • •	A slow, memory-hard hash
GPU dictionary attack on a leak	• • • •	Origin-bound hardware keys
Credential stuffing from other breaches	• • • •	Unique salts
Real-time phishing proxy	• • • •	A second factor

Answers · for the lecturer

Hashing, passwords and authentication

Cyber security · University · Lecture 3

1 **A FIDO2 security key**

A FIDO2 key signs only for the real site's origin; an app code or a push approval can be relayed by the fake page or approved by a tricked user.

2 **A cracker tries 10^9 fast hashes per second. If each Argon2id guess costs 10^6 times more, it manages 1000 guesses per second. A 10^7-word dictionary then takes 10000 seconds per account.**

$10^9 / 10^6 = 10^3$ guesses/s, and $10^7 / 10^3 = 10{,}000$ s per account, multiplied by every account because of salts.

3 **Argon2id with a unique salt per user; Checking new passwords against breached lists**

A shared salt and a fast hash still allow fast cracking; reversible encryption means one key leak exposes every password.

4 **True**

SMS codes help against password reuse, but a fake login page can ask for the code and use it at once.

5 **Precomputed rainbow tables — Unique salts**
GPU dictionary attack on a leak — A slow, memory-hard hash
Credential stuffing from other breaches — A second factor
Real-time phishing proxy — Origin-bound hardware keys

Each defence answers a specific attack; students should be able to say which.

For you to think about

How could your students experience the difference between fast and slow hashes safely, without real users' data?